

VIT BHOPAL

Student Attendance Manager

Java Console Application

PROJECT REPORT

Project Title: Student Attendance Manager
Project Type: Java console application
Submitted To: VITyarthi

Faculty Name: Nilesh Kunhare

Student Name: Pratham Kumar Yadav
Register Number: 25BAI11304
Institution: VIT Bhopal University

1. Introduction

Student Attendance Manager is a Java program that runs entirely in the terminal and keeps track of student attendance. You can add students, list who's registered, mark each student present or absent for a given date, and pull up a report either for one date or across every date on record. Records are kept in a plain CSV file, so the data is still there the next time you open the program.

Building it meant working with records, collections, file I/O, date parsing, exception handling, console formatting, and percentage math — enough to cover the core object-oriented ideas without reaching for anything heavier. There's no database, no build tool, no network calls, and no external libraries. Just the Java standard library.

2. Problem Statement

Attendance tracked by hand — a notebook, a spreadsheet someone forgets to update — tends to fall apart the moment you need to answer a simple question:

- Which students are actually registered in the class?
- Who showed up, and who didn't, on a given day?
- How many sessions has each student attended overall?
- What's each student's attendance percentage across everything recorded so far?

This application answers those questions with a menu-driven console tool that saves everything locally and can generate a summary report on demand.

3. Functional Requirements

ID	Requirement	Description
FR-01	Add student	Save a unique student ID and non-empty student name.
FR-02	List students	Display all registered students with their IDs and names.
FR-03	Record attendance	Save Present or Absent status for every student on a valid date.
FR-04	View report	Display present count, absent count, and attendance rate for each student.
FR-05	Date filtering	Show a report for one date or for all saved attendance dates.
FR-06	Persistent storage	Load existing records at startup and save changes to a CSV file.
FR-07	Input validation	Reject blank fields, invalid dates, and invalid attendance choices.
FR-08	Exit safely	Save the current data before closing the application.

All of this comes together in one menu:

- Add student
- List students
- Record attendance
- View attendance report
- Save and exit

4. Non-functional Requirements

- **Usability** — a numbered menu and plain prompts, so a first-time console user isn't lost.
- **Reliability** — the data directory and CSV file get created automatically the first time something is saved.
- **Validation** — student IDs and names can't be blank, dates must be valid ISO dates, and attendance entries have to be P or A.
- **Performance** — a LinkedHashMap keeps lookups fast, which is plenty for a classroom-sized dataset.
- **Maintainability** — input handling, storage, reporting, and the data models all live in one clearly organized file, which fits a project this size.
- **Portability** — standard Java APIs only, but it needs Java 16+ for records and the newer switch syntax.
- **Privacy** — attendance data stays on the local machine; nothing goes over a network.

5. System Architecture

The structure is intentionally shallow:

- **Presentation layer** — AttendanceApp.java shows the menu, reads input, prints results.
- **Domain model** — Student holds an ID and a name; AttendanceStatus holds a date, a student ID, and a present/absent flag.
- **Attendance service** — manages the student and attendance maps, applies date filters, and works out totals.
- **Persistence layer** — data/attendance.csv stores everything and gets loaded on startup.
- **Reporting layer** — calculates present count, absent count, and attendance percentage.

Data flow

```
User at terminal
→ AttendanceApp.java
→ Student records / Attendance records
→ data/attendance.csv
```

On startup, the program reads the CSV file if one exists. From there, student and attendance records live in memory while the program runs — adding a student or recording attendance rewrites the CSV immediately, and exiting saves once more before the program closes.

6. Design Diagrams

System architecture

```
User at terminal
→ AttendanceApp.java
→ Student records / Attendance records
→ data/attendance.csv
```

Application workflow

```
Start → Load CSV → Display menu
→ Add student / Record attendance / View report
→ Save changes → Return to menu → Exit
```

After every operation, control goes back to the main menu, so you can add students, record attendance, and pull reports as many times as you want in one sitting.

7. Design Decisions & Rationale

- **Console interface** — lightweight, quick to run, easy to demo during evaluation.
- **CSV storage** — gives persistence without setting up a database or JDBC. You can also just open the file and read it.
- **LinkedHashMap** — keeps students listed in the order they were added while still allowing direct lookup by ID.
- **Composite attendance key** — date + student ID together identify one record, so re-entering the same date for the same student updates it instead of creating a duplicate.
- **Immediate saving** — writes after every add or record action, so an update is rarely lost.
- **Input loops** — bad dates or attendance choices get rejected and re-prompted instead of crashing the program.
- **Percentage calculation** — present divided by total, shown to one decimal place; a student with no records gets a dash instead of a number.

8. Implementation Details

8.1 Student and attendance models

Student is a record holding an ID and a name. AttendanceStatus is a record holding a date, a student ID, and a boolean for present/absent. Records keep these as small, immutable objects, which makes the calculations built on top of them easy to follow.

8.2 Attendance persistence

Each line in the CSV file follows a fixed four-field format:

```
date,studentId,studentName,status
```

For example, here's the actual demonstration record saved by the application during testing:

```
2026-09-18,25BAI11304,Pratham Kumar Yadav,P
```

If a line in the file is malformed when it's loaded, the program skips it rather than refusing to start. Any time the data changes, the whole collection gets rewritten to the file.

8.3 Input validation

Dates are parsed with `LocalDate.parse` and have to follow `YYYY-MM-DD`. Blank student IDs or names get rejected outright. When recording attendance, only `P` or `A` is accepted — anything else and the prompt just repeats.

8.4 Report calculation

For each student, the report walks through the saved records, applies the date filter if one was given, counts present and absent entries, and works out the percentage as $\text{present} / \text{total} \times 100$.

9. Screenshots & Results

These are text captures from actual runs of the program — not mocked-up screenshots, just what the console printed.

9.1 Startup and menu

```
=== Student Attendance Manager ===
Attendance data: data/attendance.csv
1. Add student
2. List students
3. Record attendance
4. View attendance report
5. Save and exit
```

9.2 Recording attendance

```
Recording attendance for 2026-09-18
25bai10375 - rishit (P=Present, A=Absent): P
Attendance saved for 2026-09-18.

Recording attendance for 2026-09-18
25BAI11304 - Pratham Kumar Yadav (P=Present, A=Absent): P
Attendance saved for 2026-09-18.
```

9.3 Report output

```
Attendance report
ID Name Present Absent Rate
-----
25bai10375 rishit 1 0 100.0%
```

25BAI11304 Pratham Kumar Yadav 1 0 100.0%

9.4 Persistence result

To check persistence, a fresh Java process was started after the first one exited. It loaded the saved student and attendance data from data/attendance.csv and printed the same 100.0% report — confirming the data survives between runs, not just within one session.

10. Testing Approach

Test ID	Scenario	Expected result	Status
TC-01	Compile source	No compiler errors	Passed
TC-02	Add student	Student is accepted and saved	Passed
TC-03	Record P/A values	Status is saved for selected date	Passed
TC-04	View date report	Counts and rate are correct	Passed
TC-05	Fresh-process reload	Saved records are loaded	Passed
TC-06	Invalid date	Prompt repeats with validation message	Implemented
TC-07	Invalid attendance choice	Prompt repeats until P or A	Implemented

Testing was straightforward: compile with javac, then run through the console sessions by script. The first session added a student and recorded attendance; a second, freshly started process confirmed the data had actually been read back from disk rather than just held in memory. There's no build tool or test framework here on purpose — the project didn't need one.

```
Recording attendance for 2026-09-18
25BAI11304 - Pratham Kumar Yadav (P=Present, A=Absent): P
Attendance saved for 2026-09-18.
```

```
-----
25BAI11304 Pratham Kumar Yadav 1 0 100.0%
```

11. Challenges Faced

- Finding a storage format that's simple and still keeps the date-to-student link intact.
- Making attendance persist across runs without pulling in a database or any outside dependency.
- Stopping duplicate rows for the same date and student — solved with the composite key.
- Handling bad dates or attendance entries without the program just dying.
- Working out per-student attendance rates from the same stored records without duplicating data anywhere.

12. Conclusion

Student Attendance Manager does what it set out to do: register students, take attendance by date, keep it all in a CSV file between runs, and turn it into a percentage-based report. Along the way it put Java records, maps, file I/O, date parsing, input validation, and console formatting to actual use — not as a checklist, but because the project genuinely needed each of them to work.

13. Future Enhancements

- Teacher or administrator login.
- Editing or deleting a student, and correcting attendance entries after the fact.
- Monthly and date-range reports with class-level statistics.
- Exporting formatted reports to CSV or PDF from inside the app.
- Swapping CSV storage for a database once the number of students and records grows.
- Automated unit tests, and eventually a graphical or web interface.

14. References

- Oracle Java Documentation — <https://docs.oracle.com/en/java/>
- Oracle Java I/O Tutorial — <https://docs.oracle.com/javase/tutorial/essential/io/>
- Oracle Java Collections Tutorial — <https://docs.oracle.com/javase/tutorial/collections/>
- Oracle Java Date-Time Tutorial — <https://docs.oracle.com/javase/tutorial/datetime/>